CSC 2114 Assignment 4: Weighted Campus Route Finder (GBFS and A*)

Duration: 2 hours | **Language:** Python 3 (standard library only; `heapq` is allowed) | **Submission:** MUELE **Deliverables:** (1) `<group_number-Day A/B/Eve>_route_finder.py` (2) `<group_number-Day A/B/Eve>_report.pdf`

This assignment is an extension of Assignment 3. In Assignment 3 your searches used only the road network: BFS, DFS and IDS ignored the walking distances, and UCS used them to look back at the cost already paid. Now each place also gets a position on a map, so the program can **estimate the cost still to go**. That estimate is called a heuristic, and it powers the two new strategies. The new tasks are:

- Add greedy best-first search (GBFS) and A* to your search framework, using the heuristic defined in Section 1.
- Run all six strategies on the same ten problems; log nodes expanded.
- Find a case where greedy beats A* on nodes expanded but loses on path cost.

You extend your own Assignment 3 `route_finder.py`; you do not start again.

1. The Problem

The campus, the weights, the visitor and the interface are the same as in Assignment 3: 14 schools, 4 landmarks, 24 two-way edges, and the cost of a route is the **sum of its edge weights in metres**. The optimal route is the cheapest one.

What you must keep from Assignment 3: the `Node` class (with `DEPTH`), `solution()`, `route_cost()`, the `reached` table, the single generic search loop, the goal test on removal from the frontier, alphabetical neighbour ordering, the definition of "nodes expanded", the UCS tie-break and stale-node rule, the IDS driver, the tree-like switch with the 10,000-expansion cap, and all input handling rules. Your Assignment 3 results for BFS, DFS, UCS and IDS must **not change**.

Notation used below. For a node `n` on the way to a goal `G`:

- `g(n)` is `PATH-COST`, the metres already walked from the source to `n`.
- `h(n)` is the **heuristic**: an estimate of the metres still to walk from the place in `n` to `G`.
- `f(n) = g(n) + h(n)` is the estimated total cost of the best route through `n`.

1.1 The weighted graph (unchanged)

```
WEIGHTED_EDGES = [
    ("Main Building", "Freedom Square", 120),
    ("Main Building", "Main Library", 150),
    ("Main Building", "School of Social Sciences", 200),
    ("Main Building", "School of Liberal and Performing Arts", 350),
    ("Freedom Square", "Main Library", 90),
    ("Freedom Square", "School of Law", 180),
    ("Freedom Square", "Central Teaching Facility", 220),
    ("Main Library", "School of Education", 160),
    ("Main Library", "School of Economics", 400),
    ("Central Teaching Facility", "CoCIS", 130),
    ("Central Teaching Facility", "School of Statistics and Planning", 300),
    ("CoCIS", "School of Statistics and Planning", 100),
    ("CoCIS", "School of Engineering", 250),
    ("School of Engineering", "School of the Built Environment", 90),
    ("School of Engineering", "Margaret Trowell School of Industrial and Fine Art",
140),
    ("School of the Built Environment", "Margaret Trowell School of Industrial and Fine
Art", 70),
    ("Margaret Trowell School of Industrial and Fine Art", "School of Liberal and
Performing Arts", 450),
    ("School of Education", "School of Social Sciences", 140),
    ("School of Education", "College of Natural Sciences", 300),
    ("College of Natural Sciences", "School of Food Technology, Nutrition and
Bioengineering", 380),
    ("School of Food Technology, Nutrition and Bioengineering", "School of Agricultural
Sciences", 200),
    ("School of Agricultural Sciences", "School of Veterinary Medicine", 260),
    ("School of Statistics and Planning", "School of Economics", 350),
    ("School of Economics", "School of Law", 120),
]
```

1.2 Map coordinates and the heuristic (new)

Every place has an `(x, y)` position on a flat map, in whole metres. Copy this dictionary into your program unchanged. Like the weights, the coordinates belong to a simplified teaching model and are **not survey-accurate**, so do not change them.

```
COORDINATES = {
    "Main Building": (500, 455),
    "Freedom Square": (565, 370),
    "Main Library": (610, 435),
    "Central Teaching Facility": (425, 220),
    "CoCIS": (380, 115),
    "School of Engineering": (155, 75),
    "School of the Built Environment": (80, 70),
    "Margaret Trowell School of Industrial and Fine Art": (40, 110),
    "School of Law": (700, 260),
    "School of Education": (665, 570),
    "School of Social Sciences": (560, 630),
    "School of Liberal and Performing Arts": (165, 520),
    "School of Economics": (720, 155),
    "School of Statistics and Planning": (425, 40),
    "College of Natural Sciences": (860, 745),
    "School of Food Technology, Nutrition and Bioengineering": (1065, 990),
    "School of Agricultural Sciences": (1170, 1115),
    "School of Veterinary Medicine": (1310, 1275),
}
```

Definition. The heuristic is the straight-line distance from a place to the goal, rounded **down** to a whole number of metres:

```
import math

H_SCALE_PERCENT = 100   # leave at 100 for submission; Part C changes it

def h(place, goal):
    (x1, y1), (x2, y2) = COORDINATES[place], COORDINATES[goal]
    return math.isqrt((x1 - x2) ** 2 + (y1 - y2) ** 2) * H_SCALE_PERCENT // 100
```

`math.isqrt` gives the exact integer square root, so every student's `h` values agree and ties are real ties. `h(goal, goal)` is 0.

Worked example. From Main Building to School of Law the coordinates differ by 200 m east and 195 m north, so `h = isqrt(200² + 195²) = 279`. The cheapest actual walk (Main Building, Freedom Square, School of Law) costs 120 + 180 = 300 m. The estimate is a little low, which is exactly what a good heuristic should be.

Why this heuristic is safe. In this model every edge weight is **at least** the straight-line distance between its two places (you will verify this in Part B), because a real path can never be shorter than a straight line. Some edges are much longer than the straight line: Central Teaching Facility to School of Statistics and Planning is 300 m on foot but only 180 m on the map. Those are the places where the heuristic can mislead a search.

Rules for using the heuristic

- `h` and `COORDINATES` may be read **only** by GBFS and A* (their priority keys and the `Heuristic at source` output line) and by the checking code in B1. They **must not** influence BFS, DFS, UCS or IDS.
- Your search code must receive the graph, the coordinates and the goal as arguments and must not read the module-level dictionaries directly. Marking includes runs on a modified copy of the coordinates.
- Edge weights keep the role they had in Assignment 3.

2. Practical Session Plan

Part	Task	Time
A	Formulate the informed-search problem	15 min
B1	Heuristic function and its checks	10 min
B2	GBFS and A* in the generic loop	20 min
B3	Interface, `--batch` and `--scan`	15 min
B4	Verify against the reference values	15 min
C	Experiments and results	30 min
	Report writing	remaining time

Part A: Formulate (15 min)

The five problem components (initial state, actions, transition model, goal test, step and path cost) are the same as in Assignment 3, so do not rewrite them. Instead answer each of the following in two or three sentences.

1. **Evaluation functions.** Write `f(n)` for UCS, GBFS and A* in terms of `g(n)` and `h(n)`. For each, say what information about the walk it uses (cost so far, estimated cost to go, or both).
2. **Admissible.** Define an admissible heuristic. Explain why the straight-line distance is admissible for this problem. Which property of the weights guarantees it?
3. **Consistent.** Define a consistent heuristic. Show, using the same property of the weights and the triangle inequality, that `h` is consistent on every edge. Say what consistency gives A* when it uses a `reached` table.

4. **Compute by hand.** Using the coordinates, compute `h(School of Education, School of Veterinary Medicine)`. Compare it with the cost of the optimal route you obtained with UCS in Assignment 3, and state the gap.
5. **The goal changes the heuristic.** `h` depends on the goal as well as on the place. Explain why one fixed table of `h` values per place would not work for this assignment.
6. **Predict.** Without running anything: can GBFS return a route that is not the cheapest? Give a one-sentence reason in terms of what GBFS ignores.

Part B: Build and Search (60 min)

Extend your Assignment 3 `route_finder.py`. **Do not use `networkx` or any search or graph library.** The standard-library `heapq` module is allowed.

B1. The heuristic and its checks (10 min)

- Add `COORDINATES`, `H_SCALE_PERCENT` and `h(place, goal)` exactly as in Section 1.2.
- Add `check_heuristic(graph, coordinates)` that does two things and returns the counts:

  1. **Consistency.** For every goal, and for every directed edge `u -> v` (each two-way edge counts in both directions), test `h(u, goal) <= weight(u, v) + h(v, goal)`.

  2. **Admissibility.** For every ordered pair of distinct places `(place, goal)`, test `h(place, goal) <=` the cost returned by your UCS from `place` to `goal`.
- With the flag `--check-heuristic` the program prints these two lines and exits:

```
Consistency: <checks> checks, <violations> violations
Admissibility: <pairs> pairs, <violations> violations
```

B2. GBFS and A* in the generic loop (20 min)

Keep **one** search loop. The new strategies must not add `if algorithm == ...` branches to it. Give the best-first frontier a **priority key function** that receives a node and the goal, and make UCS, GBFS and A* differ only in that function and in the `reached` rule:

Algorithm	Frontier	Heap entry (smallest first)	Add a child to the frontier when
BFS	queue (FIFO)	not applicable	its state is not in `reached`
DFS	stack (LIFO)	not applicable	its state is not in `reached`
UCS	priority queue	`(g, state_name, counter, node)`	its state is not in `reached`, or it reaches the state with a lower `g` than the node stored there
IDS	stack inside a driver loop	not applicable	its state is not on the path from the root
GBFS	priority queue	`(h, state_name, counter, node)`	its state is **not in** `reached`; a state is never replaced
A*	priority queue	`(g + h, state_name, counter, node)`	its state is not in `reached`, **or** it reaches the state with a lower `g` than the node stored there

Rules for the new strategies (these are marked)

- `counter` comes from `itertools.count()`, as for UCS. The tie-break is therefore: smallest key, then alphabetically first state name, then earliest inserted.
- Apply the goal test **when a node is removed** from the frontier, as always. For A* this is what makes the answer optimal.
- A* uses the **stale-node rule from UCS**: when a removed node has a `g` greater than the `g` stored in `reached` for its state, discard it silently and do not count it as expanded. GBFS never replaces a state, so it never has stale nodes.
- "Nodes expanded" has the Assignment 3 meaning: removed from the frontier, not the goal, children generated.
- With the tree-like switch (`--no-reached`) GBFS and A* keep every generated node and discard nothing, exactly like UCS. The 10,000-expansion cap applies to all of BFS, DFS, UCS, GBFS and A*.
- Neither GBFS nor A* may read the edge weights except through `PATH-COST` (A*) and the step-cost function. GBFS must not use `PATH-COST` at all.

B3. User interface and batch tools (15 min)

The interactive prompts stay as in Assignment 3, but the algorithm prompt now accepts `BFS`, `DFS`, `UCS`, `IDS`, `GBFS`, `ASTAR` (also written `A*`) and `ALL`, case-insensitively with surrounding spaces ignored. `ALL` now runs **six** algorithms in the order BFS, DFS, UCS, IDS, GBFS, A*, one

block each, separated by a blank line. All input-handling rules from Assignment 3 apply (unknown place or algorithm, source equal to destination, `No route found.`).

Each solution is printed in the Assignment 3 format. For **GBFS and A*** print one extra line after `Nodes expanded:`; for IDS the extra line is the same as before:

```
Algorithm: A*
Route: <place 1> -> <place 2> -> ... -> <place n>
Cost (metres): <integer>
Hops: <integer>
Nodes expanded: <integer>
Heuristic at source: <integer>
```

Flags

Flag	Effect
`--no-reached`	Tree-like search for BFS, DFS, UCS, GBFS and A*, with the 10,000 cap.
`--trace`	For UCS, GBFS and A*, print one line per event: `push <state> g=<g> h=<h> key=<key>`, `pop <state> key=<key>`, `stale <state>` and `replace <state> g=<old> -> <new>`.
`--verbose`	As in Assignment 3: `Max frontier size`, and per-limit lines for IDS.
`--check-heuristic`	Run `check_heuristic` and print its two lines (B1).
`--batch`	Run the ten required problems (Part C) with all six algorithms and write `results_log.csv` (see below).
`--scan`	Run GBFS, A* and UCS on every ordered pair of distinct schools and print the summary below.

`--batch` writes `results_log.csv` in the current folder, one row per problem and algorithm, in problem order and then in the order BFS, DFS, UCS, IDS, GBFS, A*. That is 60 data rows under this exact header:

```
problem,source,destination,algorithm,route,cost_m,hops,nodes_expanded
```

`route` is the places joined with `->`. The ten problems are written into your program as a list of `(source, destination)` pairs; this is allowed because they are inputs, not answers.

`--scan` covers all 14 x 13 = 182 ordered pairs of distinct **schools** (direction matters, because `h` depends on the goal). It prints exactly these lines, with your own numbers:

```
Scan of 182 ordered school pairs
A* returned the UCS cost: <integer>
A* expanded fewer nodes than UCS: <integer>
A* expanded the same number as UCS: <integer>
GBFS returned a costlier route than A*: <integer>
GBFS expanded fewer nodes than A*: <integer>
GBFS beat A* on nodes but lost on cost: <integer>
Total nodes expanded (UCS / A* / GBFS): <integer> / <integer> / <integer>
A* reopened states: <integer>
Mean extra metres when GBFS is costlier: <integer>
Largest extra metres: <integer>
```

followed by one line for **every** pair counted in "beat A* on nodes but lost on cost", in alphabetical order of source and then destination:

```
<source> -> <destination>: A* <cost> m / <nodes> nodes, GBFS <cost> m / <nodes> nodes
```

"Reopened" means a child reaches a place that A* has already **expanded**, with a lower `g` than before. "Mean extra metres" is the sum of (GBFS cost minus A* cost) over the pairs where GBFS is costlier, divided by their number, rounded to a whole number.

B4. Verify (15 min)

Run the checks below and record the outcome in the report. If your numbers differ, find the bug before going further.

1. **Heuristic checks.** `--check-heuristic` must report `Consistency: 864 checks, 0 violations` (18 goals x 48 directed edges) and `Admissibility: 306 pairs, 0 violations`.
2. **Three reference pairs.**

Source	Destination	h at source	UCS: cost / hops / expanded	A*: cost / hops / expanded	GBFS: cost / hops / expanded
School of Law	School of Education	311	430 / 3 / 6	430 / 3 / 3	430 / 3 / 3
CoCIS	School of Agricultural Sciences	1274	1480 / 7 / 16	1480 / 7 / 10	1480 / 7 / 7
School of Social Sciences	School of Statistics and Planning	605	770 / 5 / 11	770 / 5 / 7	840 / 4 / 4

3. **Tree-like search.** Run UCS, GBFS and A* with `--no-reached` from **Margaret Trowell School of Industrial and Fine Art to School of Veterinary Medicine**. You should find that UCS hits the cap (it prints `Search stopped: expansion cap (10000) reached.`), A* finishes with cost 2130 after 112 expansions, and GBFS finishes with cost 2280 after 8 expansions. Explain in the report why A* succeeds where UCS gives up, and why neither the cycles nor the cap stop A*.

4. **Scan totals.** `--scan` must print: A* returned the UCS cost 182; A* fewer nodes than UCS 161; same number 21; GBFS costlier 63; GBFS fewer nodes than A* 99; GBFS beat A* on nodes but lost on cost 53; total nodes 1801 / 1008 / 731; A* reopened states 0; mean extra metres 124; largest extra metres 250.

5. **Reopening.** Your scan reports 0 reopened states. State in one or two sentences which property of `h` guarantees this, and why A* would still work if this number were not 0.

Part C: Experiments and results (30 min)

C1. The ten required problems. Every group uses these ten problems, so that results can be compared:

#	Source	Destination
1	CoCIS	School of Law
2	School of Education	School of Veterinary Medicine
3	School of the Built Environment	School of Law
4	Margaret Trowell School of Industrial and Fine Art	School of Veterinary Medicine
5	School of Economics	School of Liberal and Performing Arts
6	CoCIS	School of Statistics and Planning
7	College of Natural Sciences	School of Economics
8	School of Social Sciences	School of Engineering
9	School of Food Technology, Nutrition and Bioengineering	School of Law
10	School of Agricultural Sciences	Margaret Trowell School of Industrial and Fine Art

Run `--batch`. Report the results in two tables.

Table 1 has one row per problem and one column per algorithm, each cell holding **cost in metres / nodes expanded**:

#	BFS	DFS	UCS	IDS	GBFS	A*

Table 2 has one row per algorithm:

Algorithm	Problems solved at the optimal cost (of 10)	Total nodes expanded over the ten problems	Most nodes expanded on one problem

Use the UCS cost as the optimal cost. The full 60-row `results_log.csv` listing goes in an appendix.

C2. Greedy beats A* on nodes but loses on cost. Among your ten problems, and in the list printed by `--scan`, find cases where GBFS expands **fewer** nodes than A* but returns a **costlier** route. Choose one of the ten problems and one pair from the scan list with the largest extra metres, and for each give:

- both routes written out with their costs and node counts;

- the **first place where the two routes split**, with `g`, `h` and `f` of the competing nodes at that moment (use `--trace`);
- one or two sentences on why GBFS chose the costlier branch and why A* did not.

Problem 9 is a decoy: GBFS returns a costlier route there, but not on fewer nodes. Say in the report what GBFS and A* each expand on that problem and why it does not count as a case.

C3. How strong should the heuristic be? Change `H_SCALE_PERCENT` in your code to each value in the table, run `--scan` each time, and fill in the table. Then **set it back to 100**.

`H_SCALE_PERCENT`	Pairs where A* returned the UCS cost (of 182)	Total nodes expanded by A*	A* reopened states
0			
50			
100			
150			
200			

Check values: at 0 you must get 182, 1801 and 0, which is **identical to UCS** (why?). At 100 you must get 182, 1008 and 0.

3. Report (max 5 pages, PDF)

The full `results_log.csv` listing and pasted `--trace` output may go in an appendix that does not count toward the five pages. Everything else must fit inside them.

1. **Problem formulation.** Your Part A answers.
2. **Design.** What changed since Assignment 3: the coordinates and `h`, the priority key function, the two `reached` rules, the stale-node rule for A*, the tie-break, and the new flags. Include pseudocode or a diagram of the generic loop showing where the frontier, the key function and the `reached` rule plug in.
3. **Verification (B4).** Your check outputs against the reference values, the tree-like results with their explanation, and your answer on reopening.
4. **Results (Part C).** Tables 1 and 2, the greedy-versus-A* cases with their evidence, and the heuristic-strength table.
5. **Analysis.** The substantive part. Answer every question and support each answer with numbers from **your own** results.

1. **A* is optimal here.** State the two conditions from the lecture that make graph-search A* return a cheapest route (one on `h`, one on when the goal test happens). Use your scan (A* returned the UCS cost for every pair) and your Part A answers to explain why the conditions hold on this campus.
2. **Why greedy is fast but unreliable.** Using Table 2, the scan counts and your C2 cases, explain what GBFS gains and loses compared with A*. How many extra metres does GBFS cost on average when it is costlier, and in what fraction of pairs does that happen?
3. **Where the heuristic misleads.** Using a route from C2, point to the edge or place where the straight line was a poor guide (for example a long winding edge), and relate it to the gap between `h` and the true remaining cost.
4. **A* versus UCS.** Compare total nodes expanded by A* and UCS over the ten problems and over the scan. Find a problem where A* expands the **same** number of nodes as UCS and explain why the heuristic did not help there.
5. **Strength of the heuristic.** Using your C3 table, explain what happens to nodes expanded and to optimality as `H_SCALE_PERCENT` goes from 0 to 200. Which value of the scale gives UCS, which values keep A* optimal, and which violate a condition from Question 1? What happened to the reopened count, and why?
6. **Memory.** Using `--verbose` on problem 4, compare `Max frontier size` for BFS, DFS, UCS, IDS, GBFS and A*, and explain the ranking.
7. **Choosing a strategy.** For a campus guide app that must answer thousands of requests per second on a phone, which of the six would you choose, and what would you accept as a trade-off? Use your measurements.
8. **A better heuristic.** Suggest one way to make `h` less misleading on this campus without making it inadmissible (for example, using extra knowledge about the long edges), and say what it would cost to compute.

6. **Reflection.** In one short paragraph: what is the most surprising thing your results showed about heuristics?

4. Marking Scheme (100 marks)

Component	Marks
Part A: evaluation functions, admissible, consistent, worked example, predictions	10
Heuristic function, `COORDINATES`, `check_heuristic`, `--check-heuristic` output	8
One generic loop; GBFS and A* differ from UCS only in key function and `reached` rule; Assignment 3 behaviour unchanged	20
Correct routes, costs and expansion counts for all six algorithms (tie-break, stale nodes, tree-like mode, cap)	20
Interface, output format, `ALL` mode, flags and input error handling	8
Verification (B4): reference rows, tree-like results, scan totals, reopening	7
`--batch` log file and `--scan` summary correct	7
Results tables and Part C requirements (C1, C2, C3)	5
Analysis quality and use of evidence	12
Code clarity, comments and the report's presentation	3

Automatic deductions

- Hard-coded routes: 0 for the correctness marks.
- Use of a search or graph library: -20.
- Coordinates, `h` or edge weights used in any decision made by BFS, DFS or IDS (other than reporting cost); `h` or coordinates used in UCS: -10.
- Search functions that read the module-level graph or coordinates instead of their arguments: -5.
- Unresolved plagiarism referral.

5. Notes

- **Counts are marked exactly.** Use the node-expansion definition from Assignment 3: the goal node is not counted, stale nodes are not counted, and IDS counts expansions across all iterations. The `h` values must be the rounded-down integers from Section 1.2, or your counts will differ.
- **GBFS never replaces a state in `reached`;** A* does, like UCS. This difference is deliberate and affects the counts.
- **Leave `H_SCALE_PERCENT` at 100** in the submitted file.

- **Reuse your Assignment 3 code,** but submit one program that does all six algorithms. Keep it readable: someone should find the `Node` class, the frontier classes, the key functions, the generic loop and the IDS driver in under a minute.
- Submit only `<group_number-Day A/B/Eve>_route_finder.py` and `<group_number-Day A/B/Eve>_report.pdf`.